ASTR4004 Problem Set 3

[LJ Spriggs (u7916719)]

Note: Scripts can be additionally accessed at my github. Extension granted to midnight Oct 16, 2026.

1 The Stellar IMF

[50] Take the functional form of the IMF for the (relative) number of newborn stars dN in stellar mass bins $d\log_{10} M$ as a function of stellar mass,

$$\frac{dN}{d\log_{10}(M)} = \begin{cases} 0.093\exp\left[-\frac{(\log_{10}(M)-\log_{10}0.2)^2}{2\cdot(0.55)^2}\right], & \text{for } M \leq 1 \\ 0.041M^{-1.35}, & \text{for } M > 1 \end{cases}, \tag{1}$$

where $[M] = M_\odot$ throughout.

(a) From this equation, derive the form for $\frac{dN}{dM}$ and define $\text{IMF}(M) = \frac{dN}{dM}$.

Recalling our log laws, I derive

$$\begin{aligned} \log_{10}(M) &= \frac{\ln(M)}{\ln(10)} \\ \frac{d}{dM}(\log_{10}(M)) &= \frac{1}{\ln(10)}\frac{d}{dM}(\ln(M)) \\ \frac{d\log_{10}(M)}{dM} &= \frac{1}{M\ln(10)}, \end{aligned}$$

such that,

$$\frac{dN}{dM} = \frac{dN}{d\log_{10}(M)}\frac{d\log_{10}(M)}{dM} = \frac{1}{M\ln(10)}\begin{cases} 0.093\exp\left[-\frac{(\log_{10}(M)-\log_{10}0.2)^2}{2\cdot(0.55)^2}\right], & \text{for } M \leq 1 \\ 0.041M^{-1.35}, & \text{for } M > 1 \end{cases}.$$

Evaluating, we have

$$\text{IMF}(M) = \begin{cases} 0.040M^{-1}\exp\left[-\frac{(\log_{10}(M)+0.699)^2}{0.605}\right], & \text{for } M \leq 1 \\ 0.018M^{-2.35}, & \text{for } M > 1 \end{cases}.$$

I implement this in the python script `q1.py` as below.

```
# required imports
import sys
import numpy as np

# define function for IMF(M)
def imf(mass):

    # ensure input number/array of numbers
    try:
        m = np.asarray(mass, dtype=float)
    except (ValueError, TypeError):
        sys.exit("Error: Input must be a number or numeric array.")

    # apply Chabrier 2005 IMF
    imf = np.zeros_like(m) # array of zeros to overwrite w IMF function

    lo_m = (m <= 1) # low-mass mask for vectorised processing
    hi_m = (m > 1) # high-mass mask

    if np.any(lo_m): # run for low-mass stars
        exp_input = -1 * ((np.log10(m[lo_m]) - np.log10(0.2))**2) / 0.605
        imf[lo_m] = (0.040 / m[lo_m]) * np.exp(exp_input)

    if np.any(hi_m): # run for high-mass stars
        imf[hi_m] = 0.018 * (m[hi_m])**(-2.35)

    return imf
```

(b) Write a python script that discretises IMF (M) on a logarithmically-scaled grid of mass M from $M_{\rm min} = 10^{-2}$ to $M_{\rm max} = 10^{2}$. Logarithmically-scaled means that the grid of M is uniformly spaces in bins of $\log_{10} M$, which will then be log-spaces in M (eg, the bin width is linearly increasing with increasing M). Use a reasonable number of bins (sampling points) that captures the functional form well. Treat the bins defined this way as the bin edges throughout.

I generate the log-spaced grid using the following.

```
# define function for log-spaced IMF grid
def discrete_imf(m_min=1e-2, m_max=1e2, n_bins=50):

    # edges of uniformly-spaced grid in log-space between m_min and m_max
    # note: n_bins boxes has n_bins + 1 edges
    bin_edges = np.logspace(np.log10(m_min), np.log10(m_max), n_bins+1)

    # IMF at bin_edges masses
    imf_edges = imf(bin_edges)

    return bin_edges, imf_edges

# set no. bins param
n_bins = 50

# generate log-spaced grid with IMF values
grid_edges, imf_grid_edges = discrete_imf(m_min=1e-2, m_max=1e2, n_bins=n_bins)
print("Log-spaced grid successfully generated.")

# Log-spaced grid successfully generated.
```

(c) Plot your discretised function in a log-log plot and label the axes appropriately.

The following code is implemented in the script execution with `n_bins = 50`. The plot is illustrated in Fig. 1.

```
# required imports
import matplotlib.pyplot as plt

# define function to generate log-log plot of IMF values
def plot_imf(bin_edges, imf_edges):

    # setup plot
    plt.figure(figsize=(8, 6))

    # plot values
    plt.loglog(bin_edges, imf_edges, marker="^", markersize=2,
               ls="-", lw=0.8, c="DarkRed", label="Chabrier 2005 IMF")

    # labels
    plt.title(f"Discretised IMF (Chabrier 2005) with {len(bin_edges)-1:.0f} Bins", fontsize=14)
    plt.xlabel(r"Mass [$M_\odot$]", fontsize=12)
    plt.ylabel(r"No. Stars per Mass Bin [$M_\odot^{-1}$]", fontsize=12)

    # display details
    plt.grid(True, which="both", ls="--", alpha=0.3)
    plt.legend(fontsize=11)
    plt.tight_layout()

    # save plot
    plt.savefig("q1c_figure.png", dpi=200)
    plt.show()

# generate IMF plot
plot_imf(grid_edges, imf_grid_edges)
```

(d) Compute the mode mass of IMF (M).

I use the following code to conclude that the most frequent mass is $0.45\,M_{\odot}$.

```
# defined function to extract mode of IMF array
def max_imf(imf_array):
    index = np.argmax(imf_array)
    return imf_array[index]

# compute mode IMF
mode = max_imf(imf_grid_edges)
print(f"Most Frequent Stellar Mass: {mode:.2f} Msun")

# Most Frequent Stellar Mass: 0.45 Msun
```

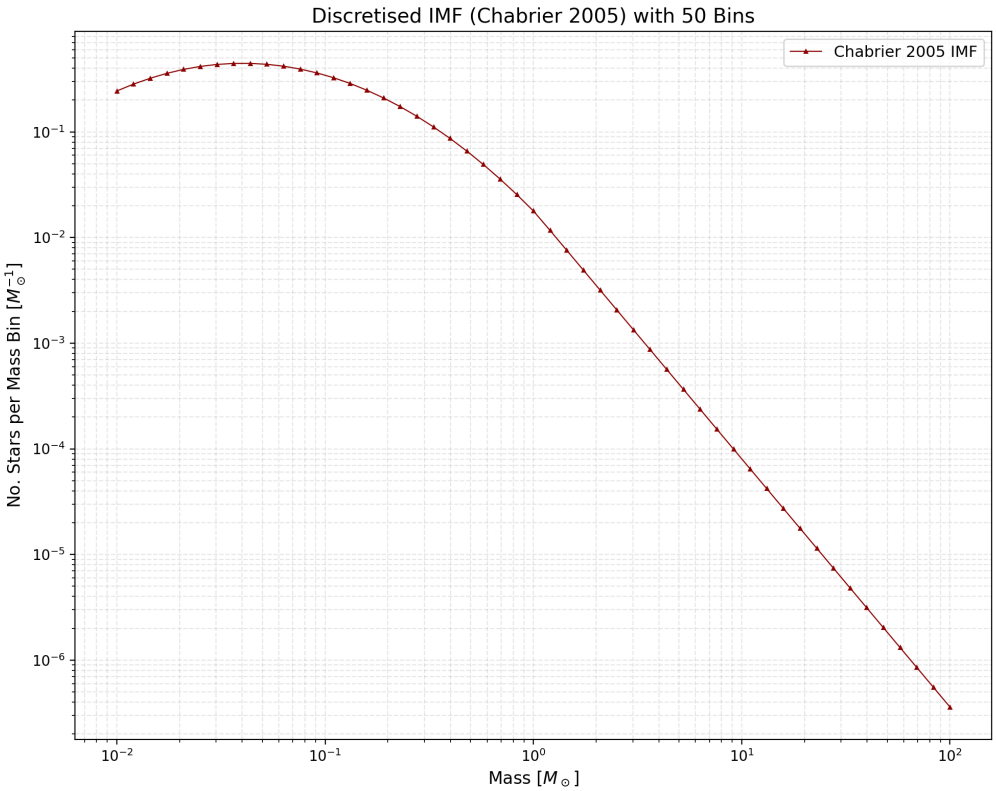


Figure 1

(e) Compute the average mass of stars from IMF (M) by numerical integration, using a sum over all bins. Implement a staggered binning approach, where the average in each bin is approximated as the arithmetic mean of the M and IMF values at the bin edges. Note that the integral over IMF (M) will not be automatically normalised to 1, so do the normalisation first (such that the function becomes a PDF) and then sum over this PDF to compute the average star mass.

As per the question, I convert the IMF into a PDF. I take the average IMF value for each bin, then multiply this by the bin's width to derive the approximate integral value corresponding to that bin (`bin_areas`). Summing all of these gives us a total area/integral with which we can normalise our IMF into a PDF (`imf_pdf`). To compute the mean, I follow

$$\mu = \int M \cdot \mathrm{IMF}_{\mathrm{norm}}(M)\, dM = \sum_{\mathrm{bins}} \langle M_{\mathrm{bin}} \rangle \cdot \langle \mathrm{IMF}_{\mathrm{norm}}(M_{\mathrm{bin}}) \rangle \,.$$

This evaluates a mean stellar mass of $0.56\, M_\odot$ (using the code below).

```
# define function to numerically compute average mass
def avg_mass(bin_edges, imf_edges):

    # extract bin widths from edges
    bin_widths = np.diff(bin_edges)

    # compute arithmetic mean for each bin
    bin_centres = 0.5 * (bin_edges[1:] + bin_edges[:-1])
    imf_centres = 0.5 * (imf_edges[1:] + imf_edges[:-1])

    # normalise IMF to PDF
    bin_areas = imf_centres * bin_widths
    integral = np.sum(bin_areas)
    imf_pdf = bin_areas / integral

    # sum to derive average mass
    avg_mass = np.sum(bin_centres * imf_pdf)

    return avg_mass

# compute average mass
avg = avg_mass(grid_edges, imf_grid_edges)
print(f"Average Stellar Mass: {avg:.2f} Msun")

# Average Stellar Mass: 0.56 Msun
```

(f) Test how the resulting average star mass depends on the number of bins. Produce a plot of the average M as a function of the number of bins. How many bins are needed to converge on the average star mass to within 1% accuracy?

I implement a binary searching method in log-space to quickly sample across a range of 10^4 bins. The "true" average mass is determined using extremely high bin resolution (5×10^4 bins); this (and the range of the binary search) can be specified at a lower value under the input parameter `low_memory_max` for operation on computers with limited memory. The bins and mass arrays are sorted before being read into `matplotlib` in a log-log plot. The code below is used for this, with the output plot given in Fig. 2. The optimal number of bins is 64, corresponding to a mass of $0.5567\,M_\odot$ (within 1% of true average mass, $0.5513\,M_\odot$).

```
# required imports
from mpl_toolkits.axes_grid1.inset_locator import inset_axes, mark_inset

# define function that converges on mass at within 1% accuracy
def find_convergent_mass(target_percent_err=1, low_memory_max=False, keep_array=False):

    # low_memory_max must be False or an integer
    assert (low_memory_max is False) or isinstance(low_memory_max, int), "Error: low_memory_max
     must be False or integer."

    # set "true" value computed with 5e4 bins for high memory
    if low_memory_max==False:
        true_bin_edges, true_imf_edges = discrete_imf(n_bins=int(5e4))
    # else use max defined by input
    else:
        true_bin_edges, true_imf_edges = discrete_imf(n_bins=low_memory_max)

    true_mass = avg_mass(true_bin_edges, true_imf_edges)
    n_true = len(true_bin_edges) - 1

    if (keep_array is True):
        n_list = []
        mass_list = []

    # initialise min and max bin numbers
    lo_n = 1
    if low_memory_max is False:
        hi_n = int(1e4)
    else:
        hi_n = int(min(1e4, low_memory_max))

    # convert to log boundaries
    lo_log = np.log10(lo_n)
    hi_log = np.log10(hi_n)

    # param to determine log-space resolution limit
    old_n = 0

    if (keep_array is True):
        test_bin_edges, test_imf_edges = discrete_imf(n_bins=lo_n)
        test_mass = avg_mass(test_bin_edges, test_imf_edges)
        n_list.append(lo_n)
        mass_list.append(test_mass)

        test_bin_edges, test_imf_edges = discrete_imf(n_bins=hi_n)
        test_mass = avg_mass(test_bin_edges, test_imf_edges)
        n_list.append(hi_n)
        mass_list.append(test_mass)

    # convert percentage errpr to fractional error
    target_frac_err = 1e-2 * target_percent_err

    # continue when the min and max number of bins are still distinct
    # implement binary searching for efficiency
    while lo_log <= hi_log:

        # test middle of testing region
        mid_log = (lo_log + hi_log) / 2.0 # central n bins in log space
        mid_n = int(np.round(10**mid_log)) # bin number must be integer

        # if rounded n bins unchanged from previous, exit loop w current params
        if mid_n == old_n:
            break
        old_n = mid_n
```

```
        # compute avg mass at mid bin size
        test_bin_edges, test_imf_edges = discrete_imf(n_bins=mid_n)
        test_mass = avg_mass(test_bin_edges, test_imf_edges)


        if (keep_array is True):
            n_list.append(mid_n)
            mass_list.append(test_mass)

        # compute fractional error
        frac_err = np.abs(test_mass - true_mass) / true_mass

        # when within 1% err, note values, test lower bin number
        if frac_err <= target_frac_err:
            # note params
            n_optimal = mid_n
            final_mass = test_mass
            # reset bin no limit
            if mid_n > 1:
                hi_log = np.log10(mid_n - 1) # search lower half of log-parameter-space
            else:
                hi_log = lo_log - 1 # break while condition

        # outside of 1% err, true higher bin no
        else:
            lo_log = np.log10(mid_n + 1)

    if (keep_array is True):
        n_array = np.array(n_list)
        mass_array = np.array(mass_list)
        sorted_indices = np.argsort(n_array)
        return n_optimal, final_mass, n_true, true_mass, n_array[sorted_indices], mass_array[
    sorted_indices]

    return n_optimal, final_mass, n_true, true_mass

# converge on 1% error mass estimate
n_optimal, final_mass, n_true, true_mass, n_array, mass_array = find_convergent_mass(keep_array
    =True)
print(f"Optimal No. Bins: {n_optimal:.0f}")
print(f"Average Mass at {n_optimal:.0f} Bins: {final_mass:.4f}")
print(f"True Average Mass at {n_true:.0f} Bins: {true_mass:.4f}")

# plot avg mass vs n bins
plt.figure(figsize=(8, 6))
plt.loglog(n_array, mass_array, marker="^", markersize=2, ls="-", lw=0.8, c="DarkRed", label="
    Approx. Avg Mass")
plt.axhline(true_mass, color="black", ls=":", alpha=0.5, label="True Avg Mass")
plt.grid(True, which="both", ls="--", alpha=0.3)
plt.title("Numerically-Approximated Average Mass vs Bin Number", fontsize=14)
plt.xlabel("No. Bins", fontsize=12)
plt.ylabel(r"Average Stellar Mass [$M_\odot$]", fontsize=12)
plt.legend(loc="lower left", fontsize=8)

# zoomed-in plot about covnergence region
ax_inset = plt.axes([0.5, 0.5, 0.35, 0.32])
ax_inset.loglog(n_array, mass_array, marker="^", markersize=3, ls="-", lw=1.0, c="DarkRed")
ax_inset.axhline(true_mass, color="black", ls=":", alpha=0.5)
ax_inset.plot(n_optimal, final_mass, marker="o", color="DarkRed", markersize=6, label="Optimal
    No. Bins")
x_min = n_optimal / 1.2
x_max = n_optimal * 1.2
mask_zoomed = (n_array >= x_min) & (n_array <= x_max)
y_min = np.min(mass_array[mask_zoomed]) * 0.98
y_max = np.max(mass_array[mask_zoomed]) * 1.02
ax_inset.set_xlim(x_min, x_max)
ax_inset.set_ylim(y_min, y_max)
ax_inset.grid(True, which="both", ls="--", alpha=0.2)
ax_inset.set_title("Convergence Region", fontsize=9)

# save plot
plt.savefig("q1f_figure.png", dpi=200, bbox_inches="tight") # alternative for tight_layout()
    when using axes
plt.show()
```

```
# Optimal No. Bins: 64
# Average Mass at 64 Bins: 0.5567
# True Average Mass at 50000 Bins: 0.5513
```

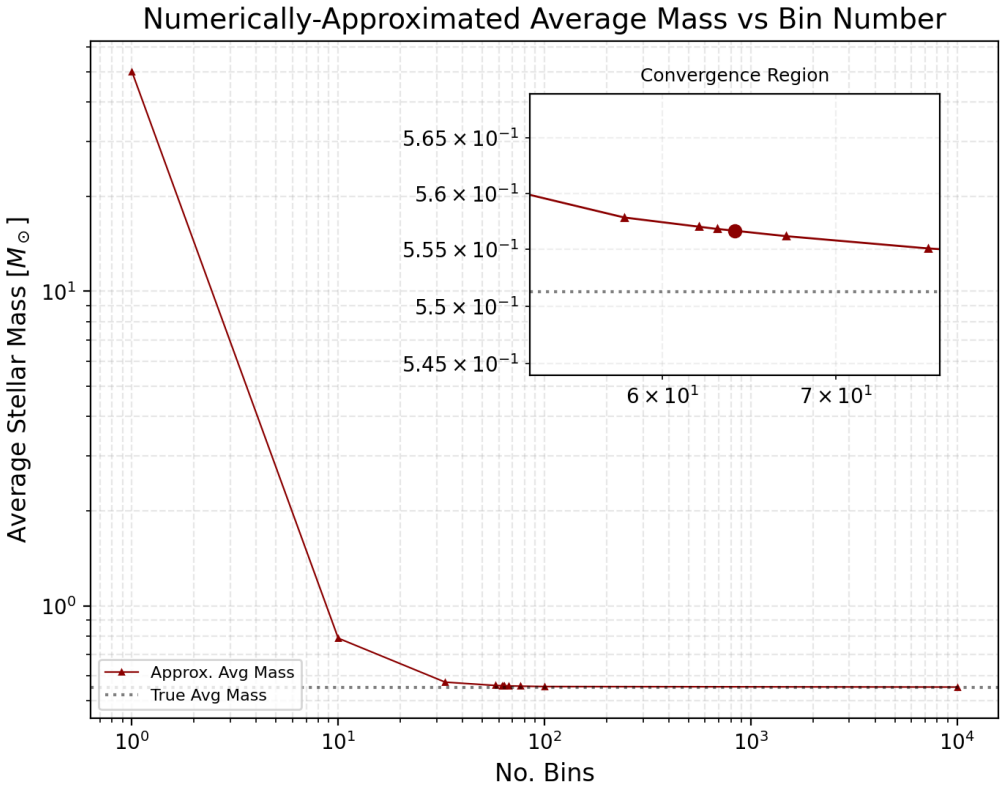


Figure 2

(g) Compute the average mass for $M_{\rm max} \to \infty$. Approximate this limit numerically by choosing a sufficiently large number for $M_{\rm max}$, such that the mean mass is converged to at least 2 significant figures. Report this $M_{\rm max}$ with 2 significant figures.

I compute the average mass for a very high $M_{\rm max} = 10^{10}$, with high bin resolution of $n_{\rm bins} = 5 \times 10^4$ (optional input argument to define lower bin number). I iterate through powers of 10 to derive the first sigfig of the lowest sufficient $M_{\rm max}$. Then, I refine this with an set of 100 equally-spaced values between the successful first sigfig of $M_{\rm max}$ and one power of 10 below. This converges at $\langle M \rangle = 0.63\, M_\odot$ for $M_{\rm max} = 9.3 \times 10^4\, M_\odot$ (see code below).

```
# define function to round any number to 2 sigfig
def round_sigfig(x, sigfig=2):
    return round(x, sigfig - 1 - int(np.floor(np.log10(abs(x)))))

# define function to increase mass until converges to 2 sigfig
def increase_mass(n_bins=5e4):

    # set no. bins param
    n_bins = int(np.round(n_bins))

    # calculate high-res mass at M_max = 1e10 for convergence comparison
    bins, imfs = discrete_imf(m_min=1e-2, m_max=1e10, n_bins=n_bins)
    ref_mass = avg_mass(bins, imfs)
    target_mass = round_sigfig(ref_mass, 2)

    # iterate through large mass jumps to determine first sigfig of M_max
    big_jump_array = 10**np.arange(1, 11, 1)
    lo_lim = None
    hi_lim = None

    for m_max in big_jump_array:
        bins, imfs = discrete_imf(m_min=1e-2, m_max=m_max, n_bins=n_bins)
        mass = avg_mass(bins, imfs)

        # check whether avg mass has converged to 2 sigfig
```

```
            if round_sigfig(mass, 2) == target_mass:
                lo_lim = m_max // 10
                hi_lim = m_max
                break

        # iterate through moderate mass jumps to find second sigfig of M_max
        mod_jump_array = np.linspace(lo_lim, hi_lim, 101)

        for m_max in mod_jump_array:
            bins, imfs = discrete_imf(m_min=1e-2, m_max=m_max, n_bins=n_bins)
            mass = avg_mass(bins, imfs)

            # check whether mean mass has converged to 2 sigfig
            if round_sigfig(mass, 2) == target_mass:
                conv_mass = mass
                final_max = m_max
                break

        return round_sigfig(conv_mass, 2), final_max

    # compute lowest convergent M_max
    conv_mass, m_max = increase_mass()
    print(f"Average mass converges at {conv_mass:.2f} Msun for mass maximum {m_max:.1e} Msun.")

    # Average mass converges at 0.63 Msun for mass maximum 9.3e+04 Msun.
```

2 Fourier Analysis

[20] Consider the figure, an image of a cloud with 1535^2 pixels (left), and the same, but smoothed (simulating telescope beam convolution) on scales of $\frac{L}{50}$ by using fourier technique (right).

(a) Consider a square image such as the one shown in the figure (left) with side length L and 1535^2 pixels. Computing a discrete 2D fourier transformation of this image, what would be the wave number range, and in what units are the wave numbers? What is the nyquist wave number in this case?

Since our image dimension is length L, the wavenumber must have units L^{-1}. For a side length of odd pixel count $N = 1535$, we get a range of

$$k \in \left[-\frac{N}{2L}, \frac{N}{2L}\right] = \left[-\frac{767}{L}, \frac{767}{L}\right].$$

For a pixel spacing of $\Delta L = \frac{L}{1535}$, the nyquist wavenumber occurs at half the corresponding frequency:

$$f_{\mathrm{Ny}} = \frac{f_{\Delta L}}{2} = \frac{1}{2 \cdot \Delta L} = \frac{767.5}{L}.$$

(b) If you have a fourier-transformed quantity q with units $[\hat{q}]$, what would the power spectrum units be?

Since a power spectrum is simply the square of the modulus of the fourier-transformed quantity, the units of the power spectrum $|q|^2$ must be $[\hat{q}]^2$. If scaled to physical units as in the figure, we will get units of $[\hat{q}]^2 L^2$ for the 2D fourier transform.

(c) What information about the original (real-space) signal is contained in the zero-wave-number bin ($k = 0$) of the power spectrum?

The central bin of a fourier-transformed signal is the mean value of the real-valued data; the central bin of the power spectrum is therefore the square of the mean.

(d) Consider again the image in the figure (left) with side length L. Suppose you want to simulate telescope beam smoothing with a beam that has a full width half maximum (FWHM) of $\sim \frac{L}{23}$, using a fourier convolution. How would you go about achieving this kind of smoothing? The result would look something like the figure (right).

The FWHM has $\mathrm{FWHM} = 2\sqrt{2\ln(2)} \cdot \sigma$, for gaussian standard deviation σ. Then,

$$\sigma = \frac{\mathrm{FWHM}}{2\sqrt{2\ln(2)}} \approx \frac{\left(\frac{L}{23}\right)}{2.355} = \frac{L}{54.17};$$

in pixels, this is $\sigma \approx 28.34$. For fourier transform $\mathcal{F}$, the convolution theorem gives us that a real-space convolution is simply a point-wise fourier-space multiplcation:

$$\mathcal{F}\left(f_{\mathrm{smooth}}\left(i, j, \sigma\right)\right) = \mathcal{F}\left(f\left(i, j\right)\right) \cdot \mathcal{F}\left(g\left(i, j, \sigma\right)\right),$$

where $f\left(i, j\right)$ is the image at pixel coordinates (i, j), $f_{\mathrm{smooth}}\left(i, j\right)$ is the smoothed image, and

$$g\left(i,\, j,\, \sigma\right) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2+j^2}{2\sigma^2}\right)$$

is a discrete gaussian kernel of standard deviation σ. The fourier transform of this has

$$\mathcal{F}(u,v) = \sum_{i,\,j} g\left(i,\, j,\, \sigma\right) \exp\left(-i\frac{2\pi}{N}(ui+vj)\right),$$

where $(u,\, v)$ are the inverse pixel coordinates in fourier space.

Practically, I use implement this using `scipy.fftpack.fft2`, which runs a fast fourier transform on the image (and `ifft2` inverts this at the end to extract the real, smoothed image). I may also use `scipy.ndimage.fourier_gaussian` to generate the fourier-transformed gaussian which can be multiplied by the fourier-transformed image.

3 Fourier Transforms & Parallel Scaling

[30] Here you will make a python program that reads a column density map of a molecular cloud near the Galactic centre, apply mirroring and zero-padding to the image, compute the fast fourier transform (FFT) with the `pyFFTW` library, make a fourier image and compute the power spectrum of the column density map.

Put everything into an executable python script that runs the entire analysis with the input file (the column density map `.fits` file) sitting in the same folder. The script should automatically produce the images (original column density image, mirrored, zero-padded, and fourier image), as well as the final plot of the column density power spectrum.

(a) Download the observational column density map from `http://www.mso.anu.edu.au/čhfeder/teaching/astr_4004_8004/material/brick.fits`. Use the `astropy` library to read the data map in the `.fits` file into a `numpy` array.

I download the data using the following code.

```
wget -O brick.fits http://www.mso.anu.edu.au/~chfeder/teaching/astr_4004_8004/material/brick.
    fits
```

I read-in the data using `astropy`, as per the question.

```
# required imports
import numpy as np
from astropy.io import fits

# read-in .fits data
# ensure file in same dir as this script
def readin_fits(filename="brick"):
    with fits.open(f"./{filename}.fits") as file:
        file.info() # print file details/structure
        data = file[0].data

    return data

filename = "brick"

# read-in data
data = readin_fits(filename="brick")
print(f"Data successfully read-in as type {type(data)} with shape {data.shape}.")

# Filename: ./brick.fits
# No.    Name      Ver    Type      Cards   Dimensions   Format
#   0  PRIMARY       1 PrimaryHDU       9   (429, 639)   float64
# Data successfully read-in as type <class 'numpy.ndarray'> with shape (639, 429).
```

(b) Make a python function to produce an image of the map with a colorbar and write the image to a `.pdf` file named `brick.pdf`. See the lefthand panel of the figure for an example of how this should look like.

I use `matplotlib.pyplot.imshow` to plot the 2D data (see code below and Fig. 3).

```
# required imports
import matplotlib.pyplot as plt

# define function for plotting simple cmap
def col_dens_map(data, filename="brick"):

    # plot setup
    plt.figure(figsize=(6, 8))
    plt.title("column density map")
```

```
    # plot data
    plt.imshow(data, aspect="auto", cmap="jet")
    plt.colorbar()

    # save plot
    plt.tight_layout()
    plt.savefig(f"./{filename}.pdf")
    plt.show()

# preliminary cmap plot
col_dens_map(data, filename="brick")
```

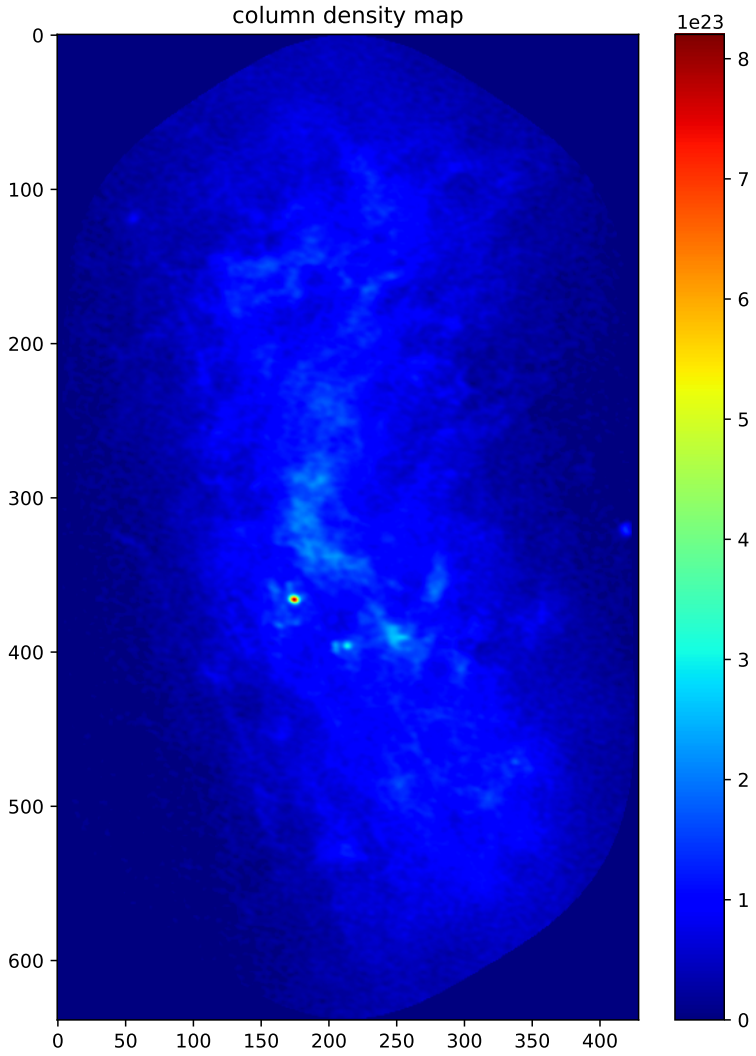


Figure 3

(c) Use the `numpy` functions `np.fliplr` and `np.flipud` to produce a mirrored array and image. Write the image to a `.pdf` file called `brick_mirrored.pdf` (see the middle panel of the figure for a thumbnail).

I use `np.block` to stack the mirrored images together (see code below and Fig. 4).

```
# define function for plotting doubly-mirrored image
def mirror_col_dens(data, filename):

    # mirrored data setup
    no_mir = np.array(data)
    hori_mir = np.fliplr(no_mir)
    vert_mir = np.flipud(no_mir)
    full_mir = np.flipud(hori_mir)

    # stitch array as in thumbnail
    stitched_arr = np.block([
```

```
        [no_mir, hori_mir],
        [vert_mir, full_mir]
    ])

    # plot
    plt.figure(figsize=(6, 8))
    plt.title("mirrored column density map")
    plt.imshow(stitched_arr, aspect="auto", cmap="jet")
    plt.colorbar()
    plt.tight_layout()
    plt.savefig(f"./{filename}_mirrored.pdf")
    plt.show()

    return stitched_arr

# mirrored cmap plot
mir_arr = mirror_col_dens(data, filename="brick")
```

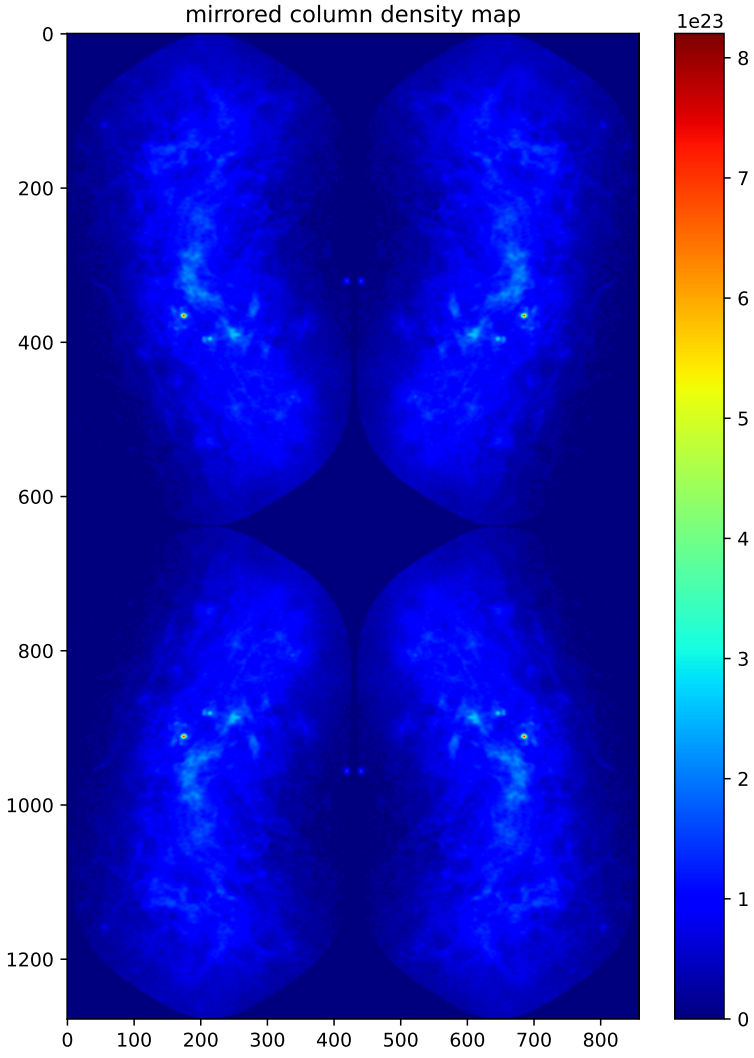


Figure 4

(d) Now use the `numpy` function `np.lib.pad` to pad zeroes symmetrically to the left and right of the image, such that the total dimensions become equal (1278, 1278). Make an image of this called `brick_mirrored_zped.pdf`.

I use `np.pad` with `constant_values = 0` (instead of `np.lib.pad`) for my python version 3.12, since this is the currently-supported function for this task. Given the prescribed dimension, I derive the required number of pixels for padding using subtraction and integer division. See code below and Fig. 5.

```
# define function to pad data to desired pixel-dimensioned square
def pad_square(data, target_dim=1278, filename="brick"):
```

```
    # derive padding values
    v, h = data.shape
    pad_v = target_dim - v
    pad_h = target_dim - h

    # handle odd values for top and bottom padding
    pad_bottom = pad_v // 2 # integer division
    pad_top = pad_v - pad_bottom

    # handle odd values for left and right padding
    pad_left = pad_h // 2
    pad_right = pad_h - pad_left

    # apply zero-padding
    padded_arr = np.pad(data, pad_width=((pad_bottom, pad_top), (pad_left, pad_right)),
                        mode="constant", constant_values=0)

    # plot
    plt.figure(figsize=(8, 8))
    plt.title("mirrored and zero-padded column density map")
    plt.imshow(padded_arr, aspect="auto", cmap="jet")
    plt.colorbar()
    plt.tight_layout()
    plt.savefig(f"./{filename}_mirrored_zped.pdf")
    plt.show()

    return padded_arr

# padded cmap plot
pad_arr = pad_square(mir_arr, target_dim=1278, filename="brick")
```

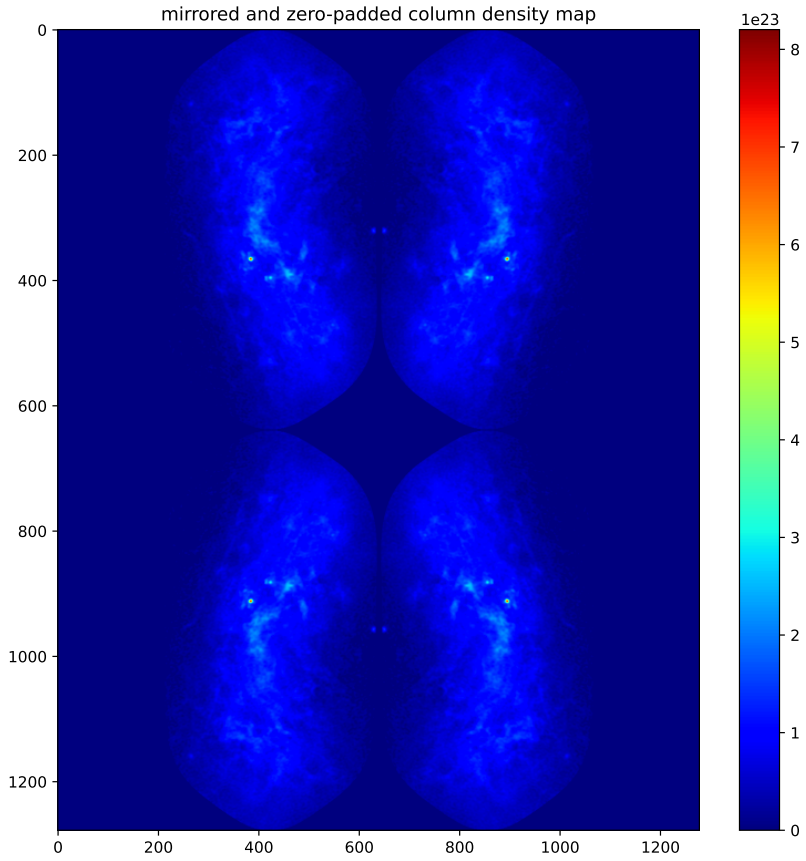


Figure 5

(e) Install `pyFFTW`. Make a 2D threaded FFTW (just use 1 or 2 threads for now) of the mirrored-and-zero-padded column density map. Shift the $\mathbf{k} = (0, 0)$ position to the centre of the fourier image and write out an image called `brick_fourier_image.pdf`. The result for this should look like the last panel of the figure.

`pyFFTW` has an implementation process that acts very similarly to `numpy`; I thus elect to use `pyfftw.builders.fft2` to compute the

fourier transform. I take the power spectrum as $P(x) = |\mathcal{F}(x)|^2$, replacing zero-values as 10^{-10} to avoid taking logs of zero. I scale the power spectrum cmap between 13 and 57 to match the provided example image. I implement the code below and produce Fig. 6.

```
# required packages
from pyfftw.builders import fft2

# define function to compute threaded FFTW
def compute_fftw(data, n_threads=2, filename="brick"):

    # compute fourier transform
    fft_builder = fft2(data, threads=n_threads)
    raw_ft = fft_builder() # execute the fourier transform

    # shift k = (0, 0) from beginning to centre of array
    shift_ft = np.fft.fftshift(raw_ft)

    # take log10 of power spectrum
    power = np.abs(shift_ft)**2
    power[power == 0] = 1e-10 # avoid log(0)
    log_power = np.log10(power)

    # plot
    plt.figure(figsize=(8, 8))
    plt.title("fourier image column density map")
    plt.imshow(log_power, aspect="auto", cmap="jet", vmin=13, vmax=57)
    plt.colorbar()
    plt.tight_layout()
    plt.savefig(f"./{filename}_fourier_image.pdf")
    plt.show()

    return shift_ft

# FFT cmap plot
fft_arr = compute_fftw(pad_arr, n_threads=2, filename="brick")
```

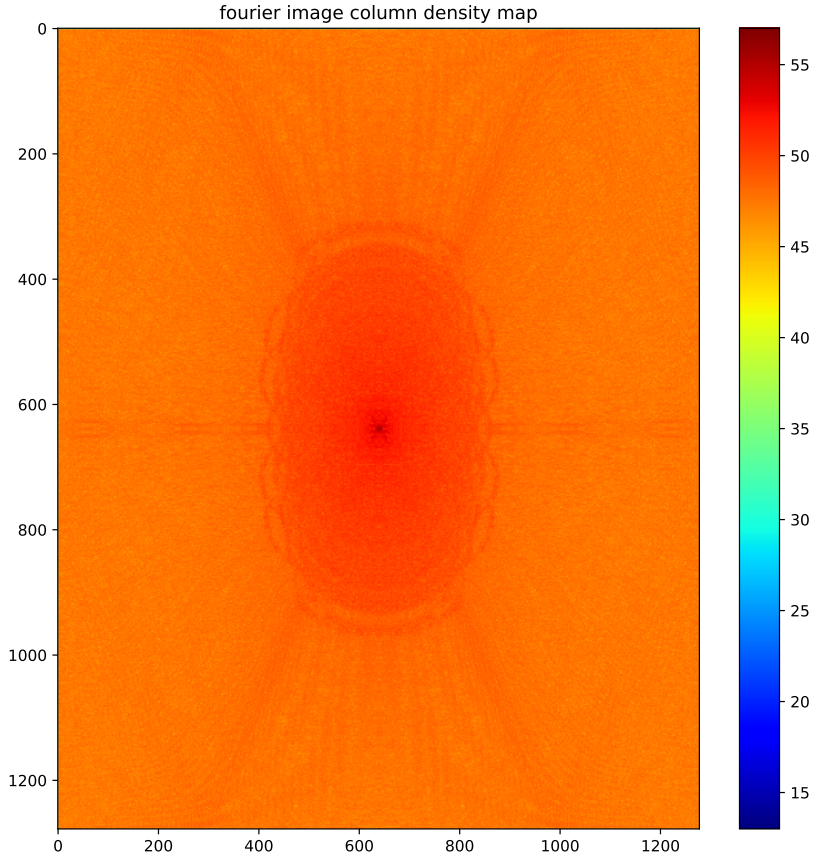


Figure 6

(f) Bin the fourier-transformed mirrored-and-zero-padded column density array $\hat{a}$ in wavenumber $k = \sqrt{k_1^2 + k_2^2}$ to obtain the power spectrum $P(k)$,

$$P(k)\,dk = 2\pi \int \hat{a}(\mathbf{k})\,\hat{a}^*(\mathbf{k})\,k dk,$$

where $\hat{a}^*$ is the complex conjugate of $\hat{a}$. Do this by defining concentric shells in wavenumber space around the centre of the fourier-transformed image with dk equal to one cell size in the fourier image. Use the following construction to get the length of the k vector in each pixel of the image (with `max_dim=1278` based on the size of the fourier image):

```
# do the k-binning
# first define a linear k array
# (same in x and y, because it's a square with max_dim^2 cells)
kxy = np.linspace(-max_dim/2, max_dim/2-1, num=max_dim)
k1 = np.zeros((max_dim,max_dim))
k2 = np.zeros((max_dim,max_dim))
for n in range(0,max_dim):
    k1[n,:] = kxy
    k2[:,n] = kxy
# define abs of k vector in each image pixel
k_image = np.sqrt(k1*k1 + k2*k2)
```

This gives you a 2D array `k_image` with the absolute values of the k vectors in each cell of the fourier image.

Now use a loop over all k values and search for the indices that fall into each shell from k to $k + dk$. sum up all the values in the fourier image with the indices just selected. Do this for each k. Here are two useful constructions to achieve this:

```
# define power spectrum and loop over all k to fill it up
Pk = np.zeros(max_dim/2+1)
k = np.linspace(0, max_dim/2, num=max_dim/2+1)
# start loop
...
    indices_in_k_shell = np.where((k_image >= ik) & (k_image < ik+1))
...
```

where the loop goes over `ik`.

I follow the instructions, where the comment in my code below outline the method of my looping through `ik`.

```
# define function to calculate 1D radially-averaged power spectrum
def radial_power_spec(ft_data, max_dim=1278, filename="brick"):

    # setup linear k array as per assignment instructions
    kxy = np.linspace(-max_dim/2, max_dim/2-1, num=max_dim)
    k1 = np.zeros((max_dim, max_dim))
    k2 = np.zeros((max_dim, max_dim))
    for n in range(0, max_dim):
        k1[n, :] = kxy
        k2[:, n] = kxy
    k_image = np.sqrt(k1*k1 + k2*k2)

    # define empty power spectrum to overwrite
    n_bins = int(max_dim // 2 + 1) # integer division
    Pk = np.zeros(n_bins)
    k = np.linspace(0, max_dim/2, num=n_bins)

    # power array as in part (e)
    power_2d = np.abs(ft_data)**2

    # loop over all shells
    for i, ik in enumerate(k): # take k bin index and radii distance of k
        k_shell_ind = np.where((k_image >= ik) & (k_image < ik+1)) # find indices in k shell
        Pk[i] = np.sum(power_2d[k_shell_ind]) # sum all power in k shell

    return k, Pk

# radially-averaged power spectrum
wavenum, power = radial_power_spec(fft_arr, max_dim=1278, filename="brick")
```

(g) Make a log-log plot of the power spectrum $P(k)$, and write this out as an image called `brick_power_spectrum.pdf`.

See code below and Fig. 7.

```
# define function to plot radially-average power spectrum
def plot_powerspec(wavenumber_arr, power_arr, filename="brick"):

    # ensure numpy arrays
    wavenumber_arr = np.array(wavenumber_arr)
    power_arr = np.array(power_arr)

    # plot
    plt.figure(figsize=(8, 6))
    plt.title("1D Radially-Averaged Power Spectrum")
    plt.loglog(wavenumber_arr[1:], power_arr[1:], markersize=0, ls="-", lw=0.8, c="DarkRed") #
    exclude k=0 to prevent log(0)
    plt.xlabel(r"Wavenumber $k$")
    plt.ylabel(r"Averaged Power $P(k)$")
    plt.tight_layout()
    plt.savefig(f"./{filename}_power_spectrum.pdf")
    plt.show()

# plot radially-averaged power spectrum
plot_powerspec(wavenum, power, filename="brick")
```

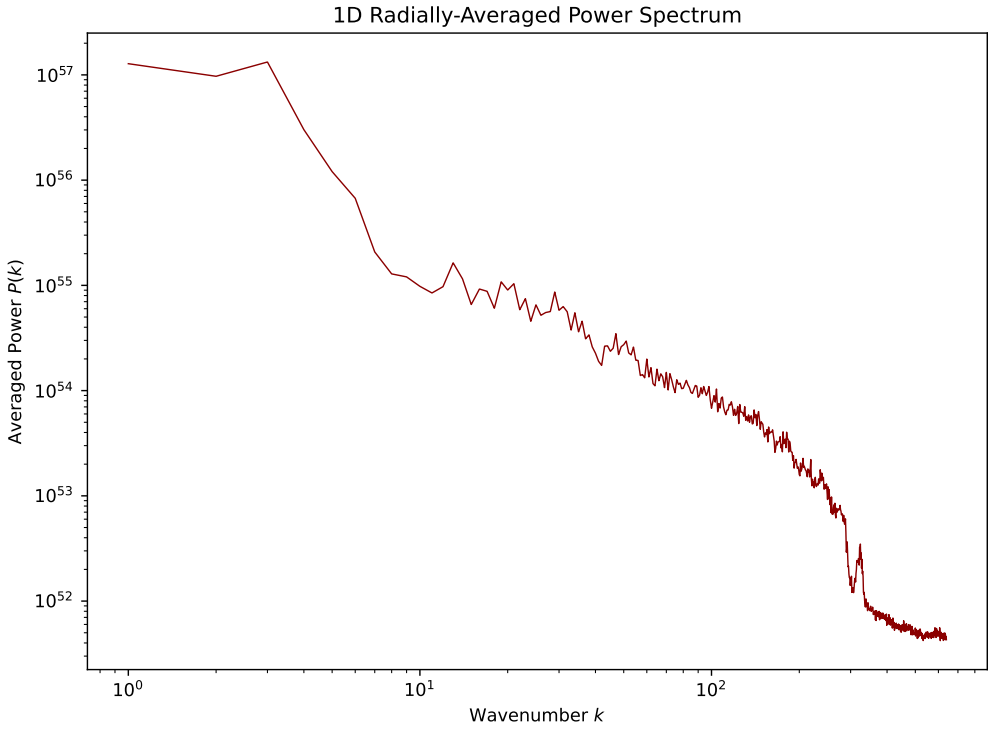


Figure 7